```
        thread_cond=&cv;
    }

    void clear_condition_variable()
    {
        std::lock_guard<std::mutex> lk(set_clear_mutex);
        thread_cond=0;
    }

    struct clear_cv_on_destruct
    {
        ~clear_cv_on_destruct()
        {
            this_thread_interrupt_flag.clear_condition_variable();
        }
    };

};

void interruptible_wait(std::condition_variable& cv,
                        std::unique_lock<std::mutex>& lk)
{
     interruption_point();
     this_thread_interrupt_flag.set_condition_variable(cv);
     interrupt_flag::clear_cv_on_destruct guard;
     interruption_point();
     cv.wait_for(lk,std::chrono::milliseconds(1));
     interruption_point();
}
```

If you have the predicate that's being waited for, then the 1 ms timeout can be completely hidden inside the predicate loop:

```
template<typename Predicate>
void interruptible_wait(std::condition_variable& cv,
                        std::unique_lock<std::mutex>& lk,
                        Predicate pred)
{
    interruption_point();
    this_thread_interrupt_flag.set_condition_variable(cv);
    interrupt_flag::clear_cv_on_destruct guard;
    while(!this_thread_interrupt_flag.is_set() && !pred())
    {
        cv.wait_for(lk,std::chrono::milliseconds(1));
    }
    interruption_point();
}
```